



DELTA: Memory-Efficient Training via Dynamic Fine-Grained Recomputation and Swapping

YU TANG, National University of Defense Technology, Changsha, China

QIAO LI, Xiamen University, Xiamen, China

LUJIA YIN, National University of Defense Technology, Changsha, China

DONGSHENG LI, National University of Defense Technology, Changsha, China

YIMING ZHANG, Xiamen University, Xiamen, China

CHENYU WANG, SenseTime, Shanghai, China

XINGCHENG ZHANG, Shanghai Artificial Intelligence Laboratory, Shanghai, China

LINBO QIAO, National University of Defense Technology, Changsha, China

ZHAONING ZHANG, National University of Defense Technology, Changsha, China

KAI LU, National University of Defense Technology, Changsha, China

To accommodate the increasingly large-scale models within limited-capacity GPU memory, various coarse-grained techniques, such as recomputation and swapping, have been proposed to optimize memory usage. However, these methods have encountered limitations, either in terms of inefficient memory reduction or diminished training performance. In response to this, our article introduces dynamic tensor offloading and recomputation (DELTA), an innovative approach for memory-efficient large-scale model training that combines fine-grained memory optimization and prefetching technology to reduce memory usage while maintaining high training throughput concurrently. Initially, we formulate the problem of memory-throughput joint optimization as an easy-solving 0/1 Knapsack problem. Leveraging this formalization, we use an improving polynomial complexity heuristic algorithm to address the problem effectively. Furthermore, we introduce, to the best of our knowledge, a novel bidirectional prefetching technology into dynamic memory management that significantly accelerates the model training when compared to relying solely on recomputation or swapping. Finally, DELTA offers users an automated training execution library, eliminating the need for manual configuration or specialized expertise. Experimental results demonstrate the effectiveness of DELTA in reducing GPU memory consumption. Compared to state-of-the-art methods, DELTA achieves substantial memory savings ranging from 40% to 72%, while maintaining comparable convergence performance for

Y. Tang and Q. Li equal contribution.

This work is sponsored in part by the National Natural Science Foundation of China under Grant No. 62025208 and Xiangjiang Laboratory Fund under Grant No. 22XJ01012.

Authors' Contact Information: Yu Tang, National University of Defense Technology, Changsha, China; e-mail: tangyu14@nudt.edu.cn; Qiao Li, Xiamen University, Xiamen, Fujian, China; e-mail: qiaoli045@gmail.com; Lujia Yin (Corresponding author), National University of Defense Technology, Changsha, Hunan, China; e-mail: yinlujia10@nudt.edu.cn; Dongsheng Li (Corresponding author), National University of Defense Technology, Changsha, Hunan, China; e-mail: lds1201@163.com; Yiming Zhang, Xiamen University, Xiamen, Fujian, China; e-mail: sdiris@gmail.com; Chenyu Wang, SenseTime, Shanghai, China; e-mail: chenyuwang814@g.ucla.edu; Xingcheng Zhang, Shanghai Artificial Intelligence Laboratory, Shanghai, China; e-mail: zhangxingcheng@pjlab.org.cn; Linbo Qiao, National University of Defense Technology, Changsha, Hunan, China; e-mail: qiao.linbo@nudt.edu.cn; Zhaoning Zhang, National University of Defense Technology, Changsha, Hunan, China; e-mail: zhangzhaoning@nudt.edu.cn; Kai Lu, National University of Defense Technology, Changsha, Hunan, China; e-mail: kailu@nudt.edu.cn.



This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2024 Copyright held by the owner/author(s).

ACM 1544-3973/2024/11-ART76

<https://doi.org/10.1145/3689338>

various models, including ResNet-50, ResNet-101, and BERT-Large. Notably, DELTA enables the training of GPT2-Large and GPT2-XL with batch sizes increased by 5.5× and 6×, respectively, showcasing its versatility and practicality in enabling large-scale model training on GPU hardware.

CCS Concepts: • **Hardware** → **Memory and dense storage**;

Additional Key Words and Phrases: Dynamic tensor recomputation and swapping, Knapsack Problem, bidirectional prefetching, GPU memory management

ACM Reference Format:

Yu Tang, Qiao Li, Lujia Yin, Dongsheng Li, Yiming Zhang, Chenyu Wang, Xingcheng Zhang, Linbo Qiao, Zhaoning Zhang, and Kai Lu. 2024. DELTA: Memory-Efficient Training via Dynamic Fine-Grained Recomputation and Swapping. *ACM Trans. Arch. Code Optim.* 21, 4, Article 76 (November 2024), 25 pages. <https://doi.org/10.1145/3689338>

1 Introduction

Large-scale models like GPT [29] have made great progress recently in plenty of domains, such as computer vision [13, 28], natural language processing [4, 46], and machine translation [53]. However, the exponential growth in the number of parameters of large-scale models has outpaced the comparatively slower increase in GPU memory capacity, a.k.a., the *GPU memory wall* [1]. This has resulted in the **out-of-memory (OOM)** problem during large-scale model training, posing a serious challenge for the development of large-scale models. Therefore, GPU memory management has emerged as a prominent research area and attracted wide attention from researchers in deep learning [3, 55, 60].

Activations are widely recognized as the primary consumers of GPU memory during deep neural network training [61]. While using a small mini-batch or implementing local gradient accumulation can alleviate the OOM problem to a certain extent, they may lead to accuracy degradation or limit the scalability of data parallelism. In extreme cases, even training with a mini-batch size of 1 can result in an OOM. For instance, GPT-3, which boasts approximately 175B parameters, would generate around 700 GB of activations when trained with a mini-batch size of 1. This amount far surpasses the memory capacity of current single GPUs. To address this challenge, tensor parallel, and pipeline parallel were proposed to divide large-scale models into smaller components, distributing them across multiple GPUs to reduce memory consumption on each GPU. It is worth noting that while multi-GPU parallelism can alleviate memory issues in large models, it will lead to heavy communication overhead and expensive equipment costs. Therefore, orthogonal optimization techniques on single GPU memory are still crucial, as they can further improve memory utilization and parallelism scalability.

To address the above issue, *recomputation* and *swapping* are two common approaches to reduce the memory footprint of activations on a single GPU. The recomputation approaches evict activations in the forward process and recomputing them in the backward process, including Checkpoint [7] and **Dynamic Tensor Rematerialization (DTR)** [21], and so on. For example, DTR targets tensors and adopts a dynamic heuristic strategy to determine which tensors should be evicted and then recomputed. However, those recomputation methods have a memory reduction upper bound that limits their memory-saving capabilities. The swapping approach exchanges activations between GPU and CPU to release the memory not in use on GPU. Compared to recomputation methods, there is no upper bound limitation on memory reduction. However, it will introduce severe delays and result in low training efficiency. Capuchin [38] focuses on the tensor-based combination of recomputation and swapping. Compared to the coarse-grained methods, which perform layerwise recomputation and layerwise swapping, fine-grained methods focus on tensors, providing much more optimization space [38]. However, it requires profiling

tensors' information in the first iteration to determine the time for swapping or recomputing, leading to execution failure when not satisfying the memory requirement in the initial, which goes against the idea of solving the OOM problem. Therefore, despite the efforts in optimizing GPU memory in the literature, coarse-grained approaches result in either inefficient memory reduction or training performance decrease.

In this article, we propose **dynamic tensor offloading and recomputation (DELTA)**, a novel, as far as we know, memory-efficient approach for large-scale model training that combines fine-grained memory optimization and prefetching techniques to reduce memory usage while maintaining high training throughput. The core idea of DELTA is the integration of swapping and recomputation, allowing it to surpass the upper limits of GPU memory optimization. First, we formulate the memory optimization problem into a variant of the Knapsack Problem that is easier to solve compared to the original problem [48]. Second, we combine dynamic tensor swapping and recomputation to optimize the GPU memory utilization, guided by an improved heuristic function that selects the optimal tensors for evicting or offloading from the GPU. Our dynamic approach eliminates the need to profile tensor information iteratively during model training, addressing the limitations of static methods. Besides, we design a bidirectional prefetching mechanism that collaboratively utilizes both prefetching from CPU to GPU and prefetching from GPU to CPU to maximize the overlap between computation and communication, thereby reducing the additional time overhead caused by frequent tensor swapping.

The main contributions of this work are threefold:

- (1) We provide a novel, as far as we know, formalization of the memory-throughput joint optimization problem that models the hard original problem into an easy-solving 0/1 Knapsack problem.
- (2) We propose a novel, as far as we know, bidirectional prefetching technology for dynamic memory management, significantly accelerating the training compared to using only recomputation or swapping.
- (3) We provide users with an automated training execution library, DELTA, that enables large-scale model training deployment with no extra manual setting or expertise experience. Besides, DELTA enables training large-scale models that requires plenty of GPUs with fewer GPUs as a result of remarkable memory reduction.

2 Related Work

2.1 Recomputation

Checkpoint [7] adopted manually recomputing every $\sqrt{N}$ layers, where N is the number of layers in the model. However, it requires manual division of **deep neural networks (DNNs)** into parts and relies on professional experience, limiting its applicability to non-linear networks, such as Inception [52] and ResNet [14]. Kirisame et al. suggested dynamic tensor recomputation, which finds an optimal tensor that has the minimum recomputation cost to evict in each iteration [21]. Zhang et al. proposed Coop to address the memory fragmentation with a sliding window, in which tensors are evicted [59]. Jain et al. treated fine-grained recomputation as a constrained optimization problem and discovered the best recomputation strategy for deep neural networks [18] with **Integer Linear Programming (ILP)** [30]. Different from Checkmate, Schuler et al. brought out XEngine, which schedules the model operators to heterogeneous devices by evicting and recomputing tensors [50]. They used a mixed-integer quadratic program to finish the operator scheduling on heterogeneous systems. These methods of constructing mathematical models, like Checkmate and XEngine, are NP-hard. Besides, they have poor scalability due to the limitations of mathematical models. In contrast, DELTA formalizes the memory optimization problem as the Knapsack problem, a special case of ILP. It is not only a more targeted and more heuristic approach but also

has polynomial time complexity with good scalability. Moreover, all of these recomputation methods have been proven to gain a sublinear memory cost of $\Omega(\sqrt{N})$, where N is the number of the tensors, a limitation upper bound of memory reduction [21].

2.2 Swapping

Rhu et al. proposed a memory runtime manager that makes the use of GPU and CPU memory virtual for each layer [45]. SwapAdvisor [15], consisting of a scheduler, a swap planner, a simulator, and a sample selector, took a dataflow graph as input and returned an augmented dataflow graph. SwapAdvisor proposed a fine-grained memory-optimization strategy based on this augmented dataflow graph. Buló et al. replaced ReLU [49] and Batch Normalization [16] layers with invertible variants [5]. Some methods tried to offload optimizer states and parameters, which saved significant GPU memory when training large-scale language models [41, 44]. Fastensor [58] aimed to optimize the path between NVMe SSD and GPU to accelerate the tensor transfer. However, HUVIM [8] utilizes the idle memory of neighboring GPUs as the destination for data transformation. Swapping cost directly depends on the tensor size and the bandwidth of GPU-CPU or GPU-GPU, which usually causes severe time overhead [15, 41, 44].

2.3 Combination of Recomputation and Swapping

SuperNeurons [54] employs a combination of swapping and recomputation but restricted swapping to convolution layers that could not be applied to language models. Another method, Capuchin [38], also integrates tensor swapping and tensor recomputation. This fine-grained method overcomes the optimization space limitation of those coarse-grained memory optimization methods. However, Capuchin uses the information in the first iteration and makes decisions for tensors based on the **memory saving per second (MSPS)** of each tensor. It does not consider the actual time overhead of in the training process and the neighbor tensors in the continuous release. Beaumont et al. also combine swapping and recomputation but targeted at layers in the training process, which is coarse-grained compared with fine-grained approaches [2]. Furthermore, it relies on the assumption of linearizing networks, which is a huge limitation for language models nowadays. POET [37] optimizes the memory consumption on edge devices by jointly combining recomputation and paging. They formulated a mixed-integer linear program to find the solution to their mathematical functions. Like Checkmate and XEngine, POET also lacks generalizability and fails in scalability. Nie et al. split tensors into more micro-tensors and offloaded micro-tensors to increase the utilization of PCIe and evicted micro-tensors to spare more GPU memory [33].

2.4 Other Methods

Moreover, there are other works regarding memory optimization. Three-dimensional (3D) Parallelism aims to reduce the memory redundancy from model parameters [24], which includes data parallelism [26], model parallelism [51], and pipeline parallelism [32]. They address the huge memory consumption by splitting the input data or the model weights. These approaches are orthogonal to those recomputation and offloading work. For example, Wang et al. integrated memory optimization and parallelism and gained a high throughput for transformer models [56]. Though 3D parallelism of large-scale model training has been developed [43, 51], GPU memory severely limits the training performance and subsequently the development and deployment of large models. It turns out that the GPU memory hinders the development of large models because of the *GPU memory wall* problem. Therefore, it is crucial to optimize GPU memory utilization. Gradient accumulation is another memory-saving method that splits the batch size into several micro batches and accumulates the gradients for each micro batch [27]. It uses the average gradients of

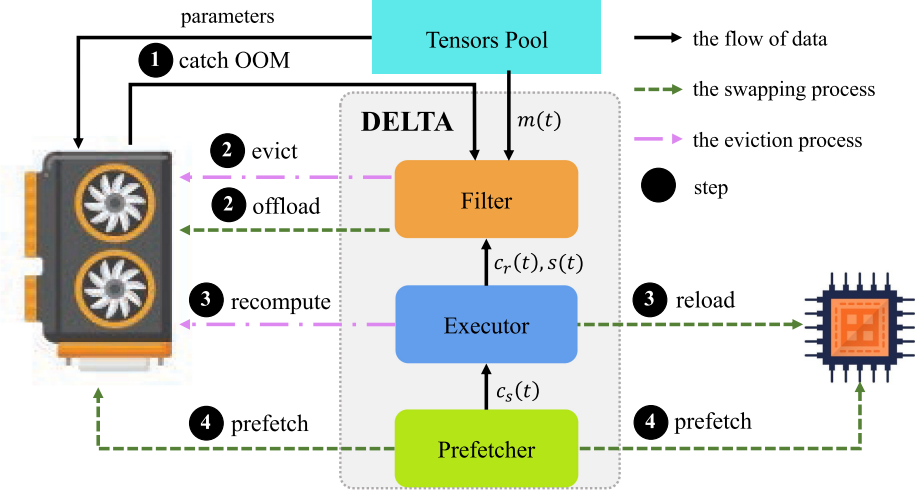


Fig. 1. The design architecture of DELTA.

all the micro batches to update the models. Gradient accumulation affects the variances of Batch Normalization, which is commonly used in vision models. These methods, although relevant, are orthogonal to our theme.

3 Overview

In this section, we introduce the design of DELTA and explain how our dynamic fine-grained optimization can achieve the goal of breaking through the upper bound of memory optimization. Figure 1 illustrates the design architecture of DELTA and the flow of tensors among components. The entire process is dynamic, eliminating the need to profile the DNN or obtain a static graph of the model. We introduce three components in DELTA, namely, Filter, Executor, and Prefetcher, for DNN training.

Filter: First, we formalize the memory management problem as a Knapsack Problem to reduce the complexity of solving the memory-throughput joint optimization problem. Under this premise, we propose a new heuristic function to manage tensors in the GPU, aiming to save GPU memory. The derived heuristic algorithm from the modeling will be integrated into Filter for determining which tensor should be performed *Evict* or *Offload* from the GPU when OOM occurs.

Executor: The main function of Executor is to promote the forward and backward propagation in DNN training. Executor uses fine-grained scheduling, in which the executed objects are tensors, not network layers. Collaborating with Filter, if a processing tensor has been evicted or offloaded, then Executor will *Recompute* it on the GPU or *Reload* it from the CPU. Besides, necessary dynamic statistics will be collected and transmitted to Filter to support its heuristic function calculation.

Prefetcher: Although it is acknowledged that swapping may introduce notable delays, we can mitigate this issue by prefetching tensors into the GPU in advance during their ancestor's recomputation, which is also called overlapping. Unlike previous works [15, 38, 41], Prefetcher uses bidirectional prefetching mechanism that not only performs *Prefetch* from CPU to GPU to overlap the delay caused by reloading but also performs *Prefetch* from GPU to CPU to overlap the delay caused by offloading.

We provide an example in Figure 2 to illustrate how DELTA solves the OOM problem. Here, t_0, t_1, t_2, t_3, t_4 , and t_5 refer to forward tensors, followed by the subsequent $t'_0, t'_1, t'_2, t'_3, t'_4$, and t'_5

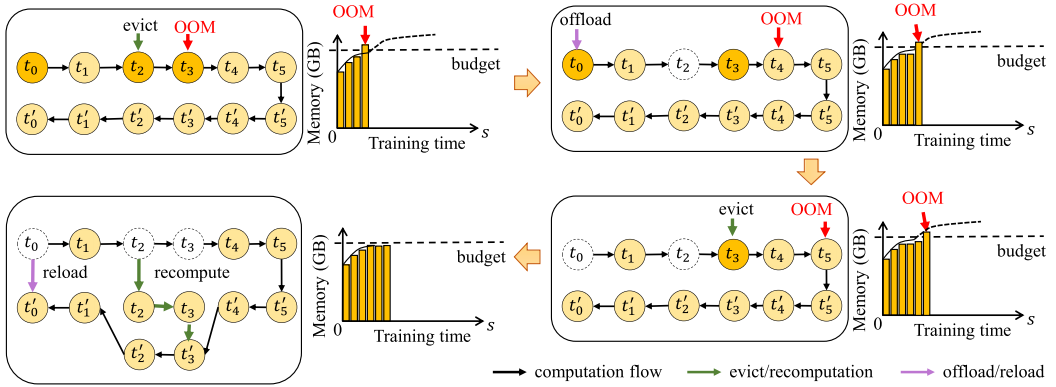


Fig. 2. Illustrations of the process of vanilla, recombination, and DELTA. The dashed lines and dotted lines symbolize memory-swapping processes and recombination steps, respectively. Different shades of color indicate the magnitude of the heuristic function value. The deeper the color the larger the value.

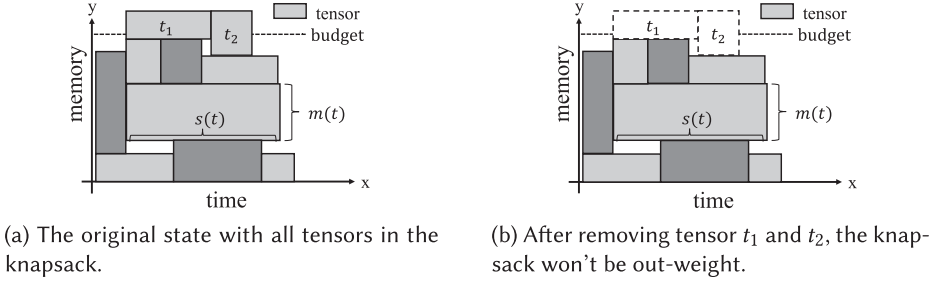


Fig. 3. Illustration of GPU memory optimization as a variant of Knapsack Problem. The rectangle's length, width, and gray level represent the staleness time, memory consumption, and cost of a tensor, respectively. We continuously remove some heavy tensors from the knapsack to ensure the knapsack is not overweight.

representing backward tensors. At first, the training process encounters OOM issues due to memory constraints, preventing it from completing successfully. Only t_0, t_1, t_2 , are retained in the GPU. The training process fails when computing t_3, t_4 , and t_5 could not fit into the GPU memory, not to mention the backward process. Then, t_2 is chosen to be evicted to reduce the memory consumption because its heuristic value is the largest before t_3 . However, OOM still exists when computing t_4 . As a result, t_0 is offloaded. If t_0 is evicted, then it cannot be reinstated on the GPU through recombination in the backward. To proceed, when OOM occurs at t_5 , t_3 is supposed to be evicted like t_2 to meet the requirement of memory budget. During the backward process, t_2 and t_3 are both reintegrated into the GPU through recombination. We need to recompute t_2 first because t_3 depends on t_2 . t_0 would be reloaded to GPU to compute the gradients and update the parameters. In all, DELTA completes the training process through a combination of dynamic recombination and dynamic swapping.

4 Formalization and Modeling

In this section, we formulate the original hard-solving problem of memory-throughput joint optimization as an easy-solving 0/1 Knapsack problem, as shown in Figure 3. The core idea is to consider the memory usage in GPU as a production of memory space and memory time.

4.1 Memory-Throughput Joint Optimization

We first introduce the relative algebraic symbols and definitions in the optimization model and refer to Figure 1. For each tensor t in the tensor set T , it has a memory consumption, $m(t)$, recomputation cost, $c_r(t)$, which is the time cost of computing this tensor and its ancestor tensors if they have been evicted, and swapping cost, $c_s(t)$, which is the total time cost of offloading and reloading this tensor, staleness time $s(t)$ that begins from $u_0(t)$ in GPU. The objective is to minimize the total extra cost C produced by recomputation and swapping. The restriction is that the total memory consumption for tensors in the GPU at any given time cannot exceed the memory budget of B . Thus, we have the optimization model Θ :

$$\Theta \left\{ \begin{array}{l} \min_{L_r \subset T, L_s \subset T} C = \sum_{t_r \in L_r} c_r(t_r) + \sum_{t_s \in L_s} c_s(t_s), \\ \text{s.t.} \quad \sum_{t \in T} \omega(t, u) \leq B, \quad u \in [S_{begin}, S_{end}], \\ L_r \cap L_s = \emptyset, \end{array} \right. \quad (1)$$

where L_r and L_s are the decision variables, representing the recomputation tensor set and the swapping tensor set, respectively. It is obvious that a tensor is a reserving tensor if it is not in $L_r \cup L_s$. We use $\omega(t, u)$ to represent the memory consumption of tensor t at time u from the iteration starting time S_{begin} to the iteration ending time S_{end} , which satisfies:

$$\omega(t, u) = \begin{cases} m(t), & u_0(t) \leq u \leq u_0(t) + s(t), \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

4.2 Knapsack Problem Modeling

The original optimization model Θ in Equation (1) is hard to solve. First, since each tensor has three options to select which set to be put in, the solution space complexity of Θ is $O(3^{|T|})$. Second, the values of $s(t)$ rely on actual runtime results, so it is hard to give a pre-solving estimation. To solve that, we simplify Θ by relaxing the restriction $\sum_{t \in T} \omega(t, u) \leq B$ through integrating the variable u on two sides of this inequality, simultaneously. Then, we have Equations (3)–(5):

$$\sum_{t \in T} \omega(t, u) \leq B, u \in [S_{begin}, S_{end}] \Rightarrow \int_{S_{begin}}^{S_{end}} \sum_{t \in T} \omega(t, u) du \leq \int_{S_{begin}}^{S_{end}} B du, \quad (3)$$

$$\int_{S_{begin}}^{S_{end}} \sum_{t \in T} \omega(t, u) du = \sum_{t \in T} \int_{S_{begin}}^{S_{end}} \omega(t, u) du = \sum_{t \in T} m(t) \times s(t), \quad (4)$$

$$\int_{S_{begin}}^{S_{end}} B du = B \times (S_{end} - S_{begin}) = B \times S, \quad (5)$$

where S is the iteration time from S_{begin} to S_{end} . Furthermore, since the recomputation and swapping merely occupy very short GPU time in an iteration, there is the following:

$$\forall t \in L_r \cup L_s, m(t) \times s(t) \ll B \times S, \quad (6)$$

We can further relax the restriction by neglecting the $m(t) \times s(t)$ for tensor $t \in L_r \cup L_s$ within a small range according to Equation (6), that is,

$$\sum_{t \in T} m(t) \times s(t) \leq B \times S \Rightarrow \sum_{t \in T \setminus (L_r \cup L_s)} m(t) \times s(t) \leq B \times S. \quad (7)$$

According to Equations (3)–(5) and (7), we can get a simplified approximate optimization model $\tilde{\Theta}$, that is,

$$\tilde{\Theta} \begin{cases} \min_{L_r \subset T, L_s \subset T} & C = \sum_{t_r \in L_r} c_r(t_r) + \sum_{t_s \in L_s} c_s(t_s), \\ \text{s.t.} & \sum_{t \in T \setminus (L_r \cup L_s)} m(t) \times s(t) \leq B \times S, \\ & L_r \cap L_s = \emptyset. \end{cases} \quad (8)$$

Furthermore, we use 0/1 variables $D(t)$ to replace the decision variables L_r and L_s in Equation (8), which represents whether a tensor t is selected to reserving, and then transform the minimization objective C to maximization objective C' , and have an equivalent optimization model, that is,

$$\tilde{\Theta} \begin{cases} \max_{D(t) \in \{0,1\}} & C' = \sum_{t \in T} D(t) \min(c_r(t), c_s(t)) - \sum_{t \in T} \min(c_r(t), c_s(t)), \\ \text{s.t.} & \sum_{t \in T} D(t) \times m(t) \times s(t) \leq B \times S. \end{cases} \quad (9)$$

Then, we remove the constant term in C' and replace $\min(c_r(t), c_s(t))$, $m(t) \times s(t)$, and $B \times S$ in Equation (9) to single symbols $v(t)$, $w(t)$, and P , respectively, and have

$$\tilde{\Theta} \begin{cases} \max_{D(t) \in \{0,1\}} & C' = \sum_{t \in T} D(t) v(t), \\ \text{s.t.} & \sum_{t \in T} D(t) w(t) \leq P, \end{cases} \quad (10)$$

where $\tilde{\Theta}$ is a variant of 0/1 **Knapsack Problem (KP)**. The difference between $\tilde{\Theta}$ in Equation (10) and standard 0/1 KP is that the $w(t)$ and P are variables in $\tilde{\Theta}$, since the staleness time $s(t)$ in $w(t)$ may increase due to other tensors' recomputation or swapping cost as well as S in P .

We demonstrate the principle with a GPU memory usage diagram to better understand why the memory-throughput joint optimization problem can be transformed into a KP. Since the memory resource is composed of two orthogonal dimensions, i.e., memory space and time, we can use a 2D Cartesian plane to represent the memory usage condition. As shown in Figure 3, the x -axis represents the GPU time, and the y -axis represents the GPU memory capacity. Since each unit of memory cannot be occupied by more than one tensor per unit of time, each tensor's memory usage can be represented as a non-overlapping rectangle placed on the 2D plane. Furthermore, each rectangle can be represented as an indivisible item t with the weight of $m(t) \times s(t)$ and the value of $\min(c_r(t), c_s(t))$. The whole limited plane can be represented as the knapsack with the capacity of $B \times S$. The objective is to search for an item placement scheme that maximizes C' , the total value of items reserved on the plane. In other words, the objective is to search for a tensor placement scheme that minimizes C , the total extra cost produced by the tensors being evicted or offloaded. Taking Figure 3 as an example, initially, we consider all tensors are stored in the knapsack, as shown in Figure 3(a). That exceeds what the backpack can hold, which means it is more than the memory budget. Then if we remove t_1 and t_2 , then the consumption will not outweigh the knapsack. Therefore, we try to remove some tensors from the GPU memory to gain a complete training process under the constraint of the memory budget and with the least time delay.

4.3 Heuristic Function

The 0/1 KP is a famous and classical ILP problem and has been proven NP-hard [47] as well as most of its variants. The most commonly used KP-accurate solutions include Dynamic Programming and Branch-and-Bound but have exponential complexity [10, 31]. Besides, there are many

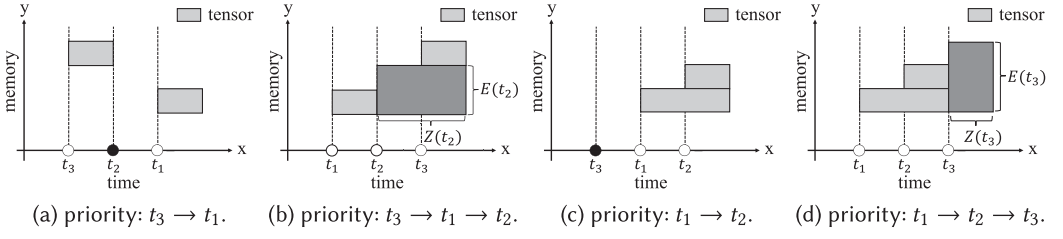


Fig. 4. Illustrations of weight compensation in Equation (12). The black button means the tensor is pinned and has no weight compensation while the white ones mean the tensor is evicted before. Here, tensor t_1 is the parent of t_2 and t_2 is the parent of t_3 . It is worth noting that the order of tensor reloading is determined by their recomputation dependency rather than their opposite order.

approximate algorithms based on heuristics. However, these methods only work when the weight of items is constant. Thus, we use the dynamic profiling of staleness time $\hat{s}(t)$ for each tensor in the recent iteration to estimate $s(t)$. Next, we use Fractional KP [17] greedy approach by continuously selecting the priority item with the highest value per unit weight to remove from the knapsack. Since each tensor has two removing operation choices, we calculate the unit value per weight of the two operations respectively and select the operation type with minimum value as for the tensor. In $\hat{\Theta}$, the value per unit weight serves as a heuristic function $h(t)$ for tensor selecting priority:

$$h(t) = \min \left(\frac{c_r(t)}{m(t) \times \hat{s}(t) - \Delta(t)}, \frac{c_s(t)}{m(t) \times \hat{s}(t)} \right), \quad (11)$$

where $\Delta(t)$ is the weight compensation factor for the evicted tensor. According to the recomputation data dependency, if a tensor is evicted, then it will cause all removing-post ancestors to be delayed for $c_r(t)$, and its memory should be pinned during all its removing-post descendants. Thus, we introduce weight compensation $\Delta(t)$ to fix the weight estimation. The $\Delta(t)$ is calculated recursively by

$$\begin{cases} \Delta(t) = E(t) \times Z(t), \\ E(t) = c_r(t) + \sum_{t' \in Pa(t) \cap L_r} E(t'), \\ Z(t) = m(t) + \sum_{t' \in Child(t) \cap L_r} Z(t'), \end{cases} \quad (12)$$

where $Pa(t)$ is the parent set for tensor t , $Child(t)$ is the child set for tensor t , respectively. The weight compensation of tensor t is a production of two orthogonal parts: the compensation time $E(t)$ caused by its post-removing parent and compensation memory $Z(t)$ caused by its post-removing children. The weight compensation can be represented as a rectangle in the memory usage diagram, as shown in Figure 4. It is worth noting that although we can neglect the weight of tensors in $L_r \cup L_s$ in knapsack capacity in Equation (8), the $\Delta(t)$ is relatively large compared with $m(t) \times \hat{s}(t)$ and cannot be neglected in Equation (11).

4.4 Discussion

Many previous works have used ILP-based methods to optimize the memory-throughput joint optimization problem. Next, we will discuss the following aspects of our model and algorithm, focusing on the performance and their relationship with and differences from other ILP methods.

Computational Complexity: The complexity of our Fractional KP greedy approach is $O(N^2)$, where N is the number of tensors. In contrast, for a general integer program modeling, it has

exponential complexity [40]. Taking POET [37] as an example, it takes the memory and energy cost on the edge devices into consideration, whose constraints are more strict and complex than Θ and used Gurobi as the optimizer to solve its model, which might cause a long execution time.

Approximation Ratio: It has been proven that the approximation ratio of the Fractional KP greedy approach on 0/1 KP is 0.5 [17]. This means that even in the worst case, the greedy algorithm's solution will be no worse than twice the training time of the optimal solution. Fortunately, in real-world scenarios with a larger number of items, we often obtain much better efficiency than the worst case. It can have a much better approximation on the random dataset, especially when the capacity of the knapsack is much larger than each item's weight, which also conforms to our scenario.

Heuristic Efficiency: In past works, there are many heuristic methods were proposed to solve the problem. As a representative of them, the DTR's heuristic function $h_{DTR}(t)$, which outperforms GreedyRemat [23], MSPS [38], and LRU strategies, could be represented as follows:

$$h_{DTR}(t) = \frac{c^*(t)}{m(t) \times \hat{s}(t)}, \quad (13)$$

where $c^*(t)$ is the total recomputation cost of all its jointed evicted neighbors. Using $c^*(t)$ rather than $c_r(t)$ in Equation (13) is to penalize the long-chained evicted tensors. Nevertheless, it will cause multiple recomputation cost calculations on prior selecting tensors, leading to over-penalization on subsequent selecting neighborhood tensors. In DELTA, we set the penalization by introducing weight compensation in Equation (12). Due to the recomputation dependency, the long continuous recomputation from t_1 to t_4 may cause the OOM in the backward pass if $\sum_{i=1}^4 m(t_i)$ exceeds the GPU memory budget. In this situation, the process achieves the optimization upper bound $\Omega(\sqrt{N})$. Besides, as shown in Figure 4, whatever the selecting priority trace is, the estimation of the total weight compensation of t_1 , t_2 , and t_3 in knapsack is equal, which provides an accurate heuristic for a cluster of the evicted neighborhood.

5 Bidirectional Prefetching

This section discusses the Prefetcher and overlapping strategies in DELTA. It is worth noting that both *Evict* and *Offload* get GPU memory reduced at the expense of training time. As pointed out in Reference [38], a suitable method involving swapping and recomputation should be largely overlapped and proceed with a minor time delay. The prefetching algorithm involves reloading tensors from the offloaded queue into GPU memory when there is sufficient space available and offloading the pinned tensors to the CPU when there is going to run out of memory in the process. These two processes are managed by a CPU-FIFO-queue and a GPU-FIFO-queue, respectively. As shown in Figure 5, on the one hand, we observe that if we reload plenty of tensors into GPU from the GPU-FIFO-queue, then there is a high probability that these tensors are chosen to *Offload* again according to the *principle of temporal locality*, which incurs extra time cost beyond our purpose. To mitigate this, we set a threshold to limit the number of reloaded tensors from the GPU-FIFO queue so as not to exceed the memory limitation in the backward pass. On the other hand, to achieve full overlap between communication and computation, after the first iteration, a CPU-FIFO-queue is used to manage the offloaded tensors in the last iteration, and these tensors will be offloaded ahead of time after they are used in the process. Based on our observation that the swapping cost is higher than the recomputation cost for the same tensor, reloading these offloaded tensors only when they are about to be used would severely impact the training process. Therefore, we adopt an early start strategy for offloading tensors, which means starting the optimization process as soon as three-quarters of the memory budget is used, as depicted in Algorithm 1. This helps achieve overlapping between communication and computation. It is worth noting that, due to the

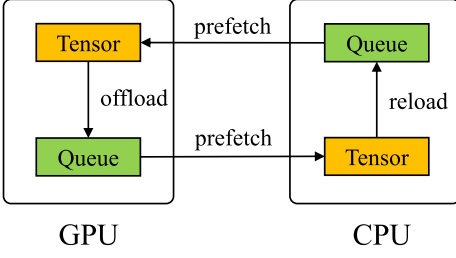


Fig. 5. The bidirectional prefetching process. There are two FIFO-queues on GPU and CPU for overlapping the delay in offloading and reloading, respectively.

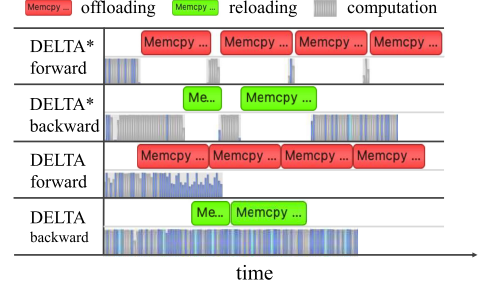


Fig. 6. The overlapping in bidirectional prefetching. The results are profiled by Nvidia's Nsight Systems [25]. DELTA* represents DELTA without prefetching.

ALGORITHM 1: The algorithm of DELTA.

```

1 Input: tensor set  $T$ , memory budget  $B$ , iteration number  $I$ ;
2 Function main()
3   for  $i = 0, 1, \dots, I - 1$  do
4      $M = 0$ ;
5      $Q_{cpu} = []$ ,  $Q_{gpu} = []$ ;
6     prefetch_shutdown = false
7   end
8   ;
9   prefetcher();
10  for  $t \in T$  being processed in propagation do
11    try
12      | executor( $t$ );
13    catch  $M \geq 0.75B$  do
14      | filter();
15    end
16  end
17  prefetch_shutdown = true;
18 end

```

recomputation time being much smaller compared to swapping time, when a tensor meets the conditions for direct recomputation (i.e., its parents are already pinned in the GPU), we will opt for recomputation instead of reloading.

Our overlapping strategies also involve asynchronously copying memory from GPU to CPU and placing the copy into a separate CUDA memory stream outside of the CUDA computation stream to finish the bidirectional prefetch process. This helps to minimize the extra time delay in the overall training process. We employ CPU multi-thread to asynchronously call CUDA stream, ensuring that the entire prefetching process follows the correct offload-reload order and avoids read-write conflicts. Thus, there are two parts to protect the thread safety in DELTA. One comes from the CUDA Stream itself to ensure the safety of computations and memory transfers on the GPU side. The other part involves the CPU side, which ensures the safety of the two prefetching queues. It is worth noting that the prefetching queue in DELTA is a type of consumer queue that can delay the consumption of tensors. We use a threshold to control the two queues' popping time. The threshold value represents the minimum number of tensors that must be pushed in the queue

ALGORITHM 2: The algorithm of Filter.

```

1  Function compute_delta(tensor t)
2      if not visit(t) then
3          visit(t) = true;
4          for  $t' \in Pa(t)$  and state( $t'$ ) = evicted do
5              compute_delta( $t'$ );
6               $E(t) = E(t) + E(t')$ ;
7          end
8          for  $t' \in Child(t)$  and state( $t'$ ) = evicted do
9              compute_delta( $t'$ );
10              $Z(t) = Z(t) + Z(t')$ ;
11         end
12          $E(t) = E(t) + m(t)$ ;
13          $Z(t) = Z(t) + c_r(t')$ ;
14          $\Delta(t) = E(t) * Z(t)$ ;
15     end
16 end
17 Function filter()
18     while  $M \geq 0.75B$  do
19         for  $t \in T$  and state(t) = pinned do
20             compute_delta(t);
21              $h(t) = \min(\frac{c_r(t)}{m(t)\hat{s}(t)-\Delta(t)}, \frac{c_s(t)}{m(t)\hat{s}(t)});$ 
22             if  $h(t) < h_{min}$  then
23                  $h_{min} = h(t)$ ;
24                  $t^* = t$ ;
25             end
26         end
27         if  $c_r(t^*)/(m(t^*) * \hat{s}(t^*) - \Delta(t^*)) < c_s(t^*)/(m(t^*) * \hat{s}(t^*))$  then
28             cuda_call.evict( $t^*$ );
29              $M = M - m(t^*)$ ;
30         else
31             cuda_call.offload( $t^*$ );
32             Q_cpu.push( $t^*$ );
33             if Q_cpu.size() > Q_cpu.threshold then
34                 Q_cpu.condition_variable.notify();
35             end
36         end
37     end
38 end

```

before popping. On the one hand, it will ensure that the tensors being offloaded and pushed in the CPU-FIFO-queue won't be immediately and pointlessly reloaded back to the GPU. On the other hand, it prevents DELTA's memory budget usage rate from being perpetually fixed below 75%, since the tensors in GPU-FIFO-queue are pinned. A result example of our overlapping strategies is depicted in Figure 6. Overall, our proposed strategies contribute to the efficient execution of DELTA and improve its training performance.

6 Implementation

The overall algorithm of DELTA is summarized in Algorithm 1. The detailed process of Filter, Executor, and Prefether modules is presented in Algorithms 2–4, respectively. These algorithms complete the overall process of DELTA regarding memory optimization. Moreover, we summarize the

ALGORITHM 3: The algorithm of Executor.

```

1 Function executor(tensor  $t$ )
2   switch state( $t$ ) do
3     case deallocated do
4       cuda_call.compute_activation( $t$ );
5        $M = M + m(t)$ ;
6       if offload  $t$  in last iter then
7          $Q_{gpu}.push(t)$ ;
8         if  $Q_{gpu}.size() > Q_{gpu}.threshold$  then
9            $Q_{gpu}.condition\_variable.notify()$ ;
10        end
11      end
12    case pinned do
13      cuda_call.compute_gradient( $t$ );
14    case evicted do
15      cuda_call.recompute( $t$ );
16      cuda_call.compute_gradient( $t$ );
17    case offloaded do
18      if state( $Pa(t)$ ) = pinned then
19        cuda_call.recompute( $t$ );
20         $Q_{cpu}.remove(t)$ ;
21        cuda_call.compute_gradient( $t$ );
22      else
23         $Q_{cpu}.wait(t)$ ;
24        cuda_call.compute_gradient( $t$ );
25      end
26    end
27 end

```

state machine of tensors in each module under different conditions in Table 1. There are four states for each tensor in the training process, namely “pinned,” “deallocated,” “evicted,” and “offloaded.” In this table, “pinned” means it is still on GPU and not deallocated from GPU. “deallocated” means the tensors get deallocated from the GPU. We also summarize the target tensors for each module. Filter processes tensors that are selected by the heuristic function $h(t)$ when facing the OOM problem. Executor conducts actions for forward tensors and backward tensors. Prefetcher prefetches tensors in the CPU-FIFO-queue or tensors in the GPU-FIFO-queue. Empirically, Filter gets called when three-quarters of the memory budget is used rather than when the allocated memory is beyond the budget to increase the overlapping degree of DELTA. The rest of the one-quarter is the reserved memory to prevent any operation from being unable if the memory were full. It is important to note that DELTA performs online optimization, ensuring memory consumption does not exceed the capacity during execution, and adheres to the original optimization model’s constraints in Equation (1). The decision to offload or recompute tensors is not determined before the training process because DELTA is an online framework. These tensors’ information is collected in each training iteration. The staleness time and the swapping cost are updated in each iteration.

The workflow is illustrated in Figure 1. First, When the program catches an OOM exception (step ① in Figure 1), Filter picks out the most suitable tensor t , from the tensors pool to perform *Evict* (the upper step ② in Figure 1) or *Offload* (the lower step ① in Figure 1). The action is determined by the comparison result of c_r and c_s , as described in Algorithm 2, if the tensor is selected to be offloaded, then it will be to the CPU-FIFO-queue (action 1 in Table 1). Otherwise, the tensor is going to be evicted (action 2 in Table 1). The weight compensation calculation in Algorithm 2 for

Table 1. The State Transition Table for Tensor's Memory Management within Each Module in DELTA

current state	condition	action	next state
Filter in OOM processing			
pinned	OOM and $c_r > c_s$	1 offload and add to CPU-FIFO-queue	offloaded
	OOM and $c_r \leq c_s$	2 evict	evicted
Executor in forward propagation			
deallocated	–	3 compute activations	pinned
pinned	offloaded in last iter	4 add to GPU-FIFO-queue	
Executor in backward propagation			
pinned	–	5 compute gradients	deallocated
evicted	parent's state = evicted	6 recompute recursively	pinned
	parent's state = pinned	7 recompute	
	parent's state = offloaded	8 reload parents and recompute	
offloaded	parent's state = pinned	9 recompute	pinned
	parent's state $\neq$ pinned	10 reload	
Prefetcher			
offloaded	tensor in CPU-FIFO-queue	11 prefetch to GPU	pinned
pinned	tensor in GPU-FIFO-queue	12 prefetch to CPU	offloaded

ALGORITHM 4: The algorithm of Prefetcher.

```

1 Function cpu_prefetch_process()
2   while not prefetch_shutdown do
3     if Qcpu.empty() then
4       | Qcpu.condition_variable.wait();
5     end
6     cuda_call.reload(Qcpu.head());
7     M = M + m(Qcpu.head());
8     Qcpu.pop();
9   end
10 end
11 Function gpu_prefetch_process()
12   while not prefetch_shutdown do
13     if Qgpu.empty() then
14       | Qgpu.condition_variable.wait();
15     end
16     cuda_call.offload(Qgpu.head());
17     M = M - m(Qgpu.head());
18     Qgpu.pop();
19   end
20 end
21 Function prefetcher()
22   thread(cpu_prefetch_process).detach();
23   thread(gpu_prefetch_process).detach();
24 end

```

each pinned candidate tensor is an $O(N)$ complexity recursive process, since each tensor will be only traversed once so that the whole complexity of the Fractional KP greedy algorithm is $O(N^2)$, where N is the number of tensors. As for the Executor, we describe the forward and backward state machine separately because the Executor performs in both forward and backward pass in Table 1. In the forward pass, computing activations will be executed when not leading to the OOM

problem for deallocated tensors(action 3 in Table 1). For pinned tensors, if they meet the condition that they are offloaded in the last iteration, then they will be added to the GPU-FIFO-queue(action 4 in Table 1). It is worth noting that the action 4 follows the action 3 immediately, so its current state is pinned. In the backward pass of Executor, the Executor checks the state of the current tensor t . If t is pinned in GPU and will not lead to OOM, then t is used to compute gradients as usual. For evicted tensors, in general, we perform *Recompute* for them (the left step ⑤ in Figure 1). More detailed actions of evicted tensors are related to their parents. If its parents are evicted, then we should recompute these tensors recursively until all of them are back to GPU (action 6 in Table 1). If its parents are pinned, then we could directly recompute this tensor (action 7 in Table 1). Otherwise, if the parents are offloaded, then we should reload their parents at first and then recompute this tensor (action 8 in Table 1). However, considering those offloaded tensors, we have two choices to get them into GPU again (action 9 in Table 1). If their parents have been pinned, then we could recompute them back to GPU and remove them from the CPU-FIFO-queue rather than reloading it (action 10 in Table 1). This helps mitigate the drawbacks of the extra time cost of swapping tensors between GPU and CPU. Besides, Executor also sends the updated $c_r(t)$, $c_s(t)$ and $\hat{s}(t)$ to Filter for the next filtering. Finally, for Prefetcher, it is an asynchronous execution process. DELTA performs *Perfetch* to GPU or CPU for those offloaded tensors from the CPU-FIFO-queue (the left step ④ in Figure 1, action 11 in Table 1) and pinned tensors from the GPU-FIFO-queue (the right step ④ in Figure 1, action 12 in Table 1), respectively. Until the prefetching is complete in GPU-FIFO-queue, the tensors have to keep memory in the GPU, so its current state is pinned. The memory contention and the thread safety issues caused by simultaneous prefetching and recomputation are guaranteed by the CUDA stream mechanism. Besides, the details for the reloading process in Executor in Line 17, Algorithm 1 is that if a tensor t is in prefetching, then the Executor will be blocking, otherwise it will remove it from the CPU-FIFO-queue and immediately reload it synchronously. In each training iteration, certain tensor information remains unchanged, such as its memory size. The staleness time is updated after each iteration is computed, which can be implemented using a C++ program. DELTA is integrated into PyTorch, modifying PyTorch's source code and executing in eager mode, consistent with PyTorch's execution mode. In theory, if using the torch.fx tool, then DELTA also supports graph mode, but the tensor information profile process is not included in DELTA's internal operations, requiring additional actions.

7 Experiments

7.1 Setup

To evaluate the performance of DELTA, we compare our method with DTR and Vanilla, which means solely using Pytorch [36] to train deep neural networks without recomputation or swapping in this article. All experiments are conducted on a cluster equipped with eight NVIDIA Tesla A100 GPUs with 80 GB memory within one node, with CUDA Toolkit 11.2 [35] and CUDNN 8.0 [34]. This system is equipped with stable PCIe 4.0, whose bandwidth is generally 64 GB/s. We train computer vision models on ImageNet [22] with Adam [20], such as ResNet-50, ResNet-101, and ViT-Large [9]. Besides, we finetune BERT-Large [19], GPT2-Large, and GPT2-XL [4] on SQuAD [42] to evaluate the efficiency of DELTA on the language models. We emphasize that we aimed to compare the performance of DELTA, DTR [21], POET [37] and Vanilla under the same conditions, including the same budget and the same hardware environment. Therefore, we ensured that the budgets of DELTA, DTR, and Vanilla were set to be identical in our experiments. Since POET is designed for edge devices, to make a fair comparison, we implemented a configuration based on the A100 GPU and conducted our experiments on this setup. Additionally, POET utilizes the Gurobi solver [12], but as the Gurobi solver has not been open-sourced, we employed the CBC solver [6] as a substitute.

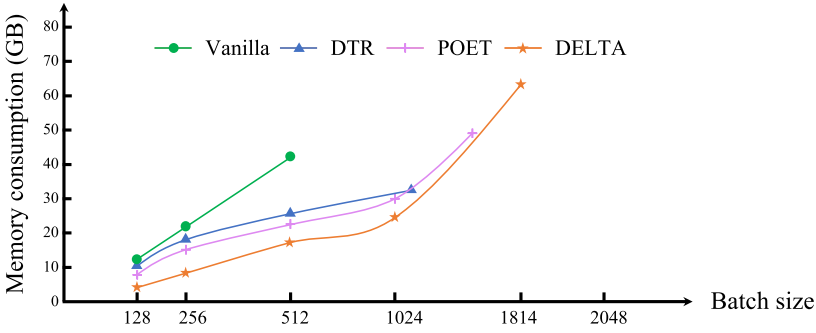


Fig. 7. The memory consumption on ResNet-50. When the batch size is larger than 1076, the Vanilla and DTR will get stuck into the OOM problem, while DELTA could fit all the batch sizes.

7.2 Memory Reduction

In this section, we present a comprehensive comparison of memory usage among different methods to evaluate the efficiency of our proposed method, DELTA. Figure 7 shows the memory consumption of ResNet-50 utilizing Vanilla, DTR, POET, and DELTA, respectively, with varying batch sizes.

We utilize `torch.cuda.max_memory_allocated` function to track the maximum memory consumption during the training process. From Figure 7, we spot that when the batch size is 1814, both Vanilla and DTR get stuck into the OOM problem, but DELTA could train ResNet-50 with 62.6 GB memory cost. Compared with DTR and Vanilla, DELTA consistently shows superior performance in terms of GPU memory savings. For instance, when the batch size is 256, DELTA trims the memory consumption from 21.3 to 8.4 GB, saving GPU memory about 60.56% for ResNet-50. Notably, DELTA achieves the maximum memory saving when the batch size is 128, reducing memory consumption from 11.2 to 4.11 GB, which translates to a substantial 63.3% reduction in GPU memory usage. In contrast, DTR only saves 6.25% from 11.2 to 10.5 GB under the same condition. It is clear that despite DTR improving the training batch size to some degree compared with Vanilla, DELTA saves more memory and achieves a larger training batch size.

In the context of optimizing GPU memory usage, we also compared DELTA with different methods, including DTR, vDNN, SuperNeurons, and Capuchin, and summarized the results in Table 2. As vDNN [45] and Capuchin [38] have not been open-sourced, we cited parts of experimental data from their article for comparison. DELTA demonstrates impressive results, achieving a memory reduction of 72.3%. Though Capuchin gains about 85% memory reduction, performing better than DELTA, it will fail when the GPU memory budget cannot satisfy its static profiling on the first iteration. This is the main drawback of these static methods, which limits the extension for model size scaling. Overall, DELTA proves to be advantageous in dynamically optimizing GPU memory usage while considering tensors' information at each iteration. It is worth noting that in DELTA, the optimization process starts when the memory consumption reaches 75% of the memory budget. However, the memory budget is a parameter provided by users. We could seek an optimal memory budget to achieve ideal memory reduction in the training process.

7.3 Maximal Batch Size

As known in References [11] and [39], a sufficiently large batch size gains promising and fast convergence. The use of smaller batch sizes due to the *GPU memory wall* problem results in prolonged training times to achieve convergent results. Hence, there is a pressing need for optimizing GPU memory in DNNs during training. This section aims to explore the maximum

Table 2. The Memory Reduction of DTR, vDNN, SuperNeurons, Capuchin, and DELTA

Methods	Category	Reduction
DTR [21]	Recomputation	40.9%
vDNN [45]	Swap	69%
SuperNeurons [54]	Combination	48.29%
Capuchin [38]	Combination	85%
DELTA (ours)	Combination	72.3%

The use of bold is to highlight the best result. “85%” is bold because it is the best. “72.3%” is bold because it is our best result.

Table 3. The Max Batch Size of Different Methods within One A100 GPU

Model	Methods	Max BS	Growth
ResNet-50	Vanilla	891	1
	DTR	1072	1.20×
	POET	1436	1.61×
	DELTA	1814	2.04×
ResNet-101	Vanilla	594	1
	DTR	761	1.28×
	POET	1024	1.72×
	DELTA	1336	2.25×
ViT-Large	Vanilla	886	1
	DTR	890	1.005×
	POET	912	1.029×
	DELTA	1662	1.86×

batch size with DELTA on ResNet-50, ResNet-101, ViT-Large, BERT-Large, GPT2-Large, and GPT2-XL. In these experiments, we utilize the full capacity of the GPU memory and summarize the results in Table 3, and Figure 8, respectively. With DTR, we could train ResNet-50 with a batch size of 1072, which is approximately 1.2× compared with the Vanilla. POET achieves maximum batch sizes of 1436 for training ResNet-50 and 1024 for training ResNet-101. As for DELTA, the maximum batch size reaches 1814 for ResNet-50, earning growth of 2.04×. DELTA behaves nearly the same in training ResNet-50 and ResNet-101. The max batch size of training ResNet-101 is 1336 using DELTA, obtaining an increment of 1.28× and 2.25× compared with Vanilla and DTR, respectively. Similarly, DELTA also increases the maximal batch size of training ViT-Large by 1.86×, reaching 1662, which outperforms DTR(1.005×) and POET(1.029×).

Turning to BERT-Large, Figure 8 presents the maximum batch size comparison among different methods. However, DELTA substantially increases the maximum batch size, from 42 to 204, representing a growth of 4.85×. As for GPT2-Large, DELTA increases the maximum batch size from 4 to 22, gaining a growth of 5.5×. In general, we could train GPT2-XL with a batch size of 2 within one A100. But we can train GPT2-XL with a batch size of 12 with DELTA, gaining a growth of 6×. Compared to DTR, whose batch size is 4, DELTA is an efficient tool for training large-scale models with large batch sizes. Technically, without using DELTA and without encountering OOM, training GPT2-Large with a batch size of 22 requires at least 6 A100 GPUs, while using DELTA only requires one. Similarly, training GPT2-Large with a batch size of 12 also requires at least 6 A100 GPUs, but using DELTA only requires one.

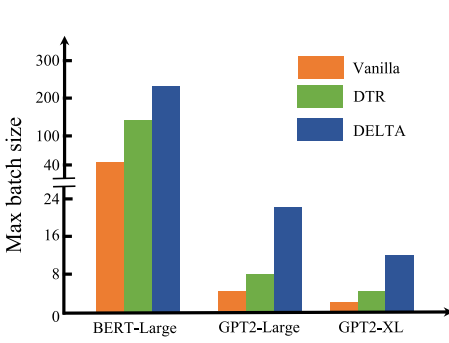


Fig. 8. The max batch size of different methods of BERT-Large, GPT2 Large, and GPT2-XL.

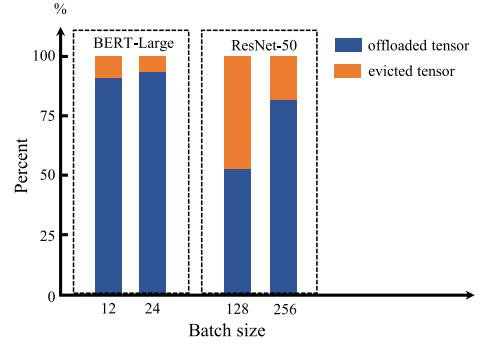
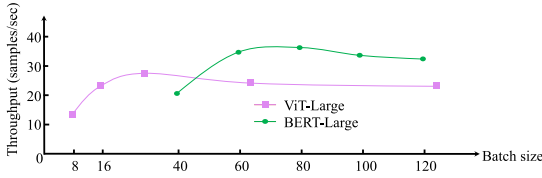
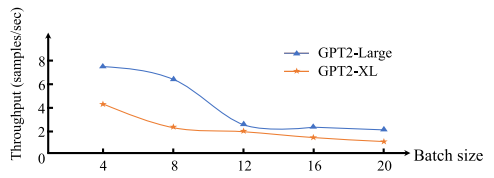


Fig. 9. The percentage of evicted tensors and offloaded tensors on ResNet-50 and BERT-Large.



(a) Throughput on BERT-Large and ViT-Large.



(b) Throughput on GPT2-Large and GPT2-XL.

Fig. 10. Throughput of DELTA on different Transformer-based models with various batch sizes.

7.4 Throughput

To delve into the throughput performance introduced by DELTA, we conduct a comparison of DELTA's throughput across some Transformer-based models as the batch size varies. These results are illustrated in Figure 10. These experiments are carried out up to the maximum batch size of the respective models. Since DELTA employs a combination of recomputation and swapping to manage GPU memory, it achieves higher training batch sizes. We observe that for ViT-Large and BERT-Large, the throughput change is relatively small as the batch size increases. Moreover, it ensures maintaining the same throughput while accommodating the increase in training batch size, thereby reducing the overall training time. For GPT2-Large and GPT2-XL, increasing the batch size results in the corresponding throughput descent. This is primarily due to an increase in offloaded tensors and the communication overhead for very large models with the batch size increasing, leading to a decrease in performance. This phenomenon can be obtained in other works, i.e., in Capuchin, when the batch size is increased from 560 to 630, the throughput will decrease merely from almost 215 to 210 samples/s in training with ResNet-50 on P100 GPU. However, this declining trend will accelerate, since when the batch size is further increased to 700, the throughput will be precipitously decreased to almost 170 samples/s. Thus, it does not recommend using a very large batch size for model training.

Besides, **gradient accumulation (GA)** is a common training method for a limited memory budget that collects gradients over several steps and updates model parameters using the average of these steps' gradients. GA can easily reduce activation memory without involving the complexity of recomputation and swapping. DELTA also provides an automatic method of reducing memory consumption in deep learning. It is worth noting that DELTA is orthogonal to gradient accumulation. Therefore, we want to explore the performance of DELTA compared to GA or combined with GA. Those results are shown in Figure 12, where the bars represent the throughput of each method,

and the points indicate the corresponding loss. “GA” is short for Gradient Accumulation and “max” means the maximum effective batch size in the vanilla process within one GPU obtained in Section 7.3. In Figure 12, we primarily compare the performance of GA and DELTA. The x -axis represents the batch size for training, while the y -axis represents the training throughput and loss. “GA, bs=1” indicates the use of the GA algorithm with a batch size of 1. In this case, the gradient accumulation steps in the GA algorithm are equal to the batch size, that is, $\text{batch size}/1 = \text{batch size}$. “GA, bs=max” represents the GA algorithm with a batch size equal to the maximum batch size used for training this model without DELTA. In this case, the gradient accumulation steps in the GA algorithm are $\text{batch size}/\text{max}$. “DELTA, bs=max” refers to training these models with DELTA with a batch size equal to the maximum batch size. “DELTA+GA, bs=max” means training these models with DELTA+GA algorithm with a batch size equal to the maximum batch size and the gradient accumulation steps remain the same as $\text{batch size}/\text{max}$. When the gradient accumulation step is 1, “DELTA, bs=max” is equivalent to “DELTA+GA, bs=max”. As the batch size increases, the gradient accumulation steps also increase. While DELTA’s throughput does not reach the levels observed with a batch size of 408, it demonstrates a considerable improvement compared to the scenario with a batch size of 1. In the case of GPT2-Large, GA and DELTA demonstrate comparable throughput at batch sizes of 22 and 88. Nevertheless, a noticeable decline in DELTA’s throughput becomes evident when the batch size is increased to 88. For GPT2-XL, we compare the results of these methods with global batch sizes of 12 and 24, respectively. These results are generally similar to those obtained with the GPT2-Large model. Technically, DELTA is orthogonal to gradient accumulation, and they could be used together to further increase the maximal batch size.

7.5 Prefetching and Overlapping

Taking all of the overlapping strategies by bidirectional prefetching into consideration mentioned in Section 5, we compare the processes of DELTA, with and without prefetching, in detail. Here, DELTA* represents DELTA without prefetching. We use Nvidia’s Nsight Systems [25] to analyze the timeline for each process within one iteration and show the results in Figure 13. Each orange block in both Figure 13 represents the computation process, the green blocks represent the tensor offloading process in the forward process, and the cyan blocks represent the tensor reloading process in the backward process. The results show that each operator proceeds in sequential order without overlapping. After we adopt prefetching, we get communication and computation overlapped in DELTA. In this way, we reduce the time by 19.7% per iteration. In practice, We notice that there are nine offloading processes while there are only two reloading processes. This is because the other seven offloaded tensors are recomputed into GPU when their parents have been pinned and the prefetching for them is skipped. Here, for better exhibition, we omit some blocks in Figure 13.

In practice, the preferred approach for handling offloaded tensors in the backward process is to perform *Reload*, as suggested in Reference [38]. We conduct various experiments on ResNet-50 with a batch size of 128 and compare the average swapping cost and recomputation cost on some familiar and typical tensors, such as *BNForward*, *ConvForward*, and *PoolingForward*. Tensors in Table 4 are usually likely to be chosen to release from GPU in our experiments. The number in “(·)” is each tensor’s corresponding unique tensor id. It turns out that for these tensors, the swapping cost is much higher than the recomputation cost, especially for *BNForward*. We also analyze the percent of evicted tensors and offloaded tensors in all the tensors and show the results in Figure 9. In Figure 9, we find that evicted tensors and offloaded tensors nearly both occupy half of the total tensors when the batch size is 128. This provides evidence that the fine-grained combination of recomputation and swapping method is more likely to swap tensors rather than evicting

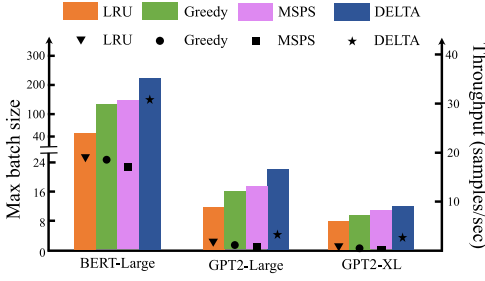


Fig. 11. Max batch size and throughput on different models and various heuristics.

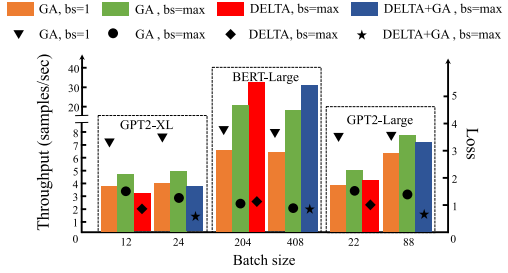


Fig. 12. Throughput and convergence comparison with gradient accumulation.

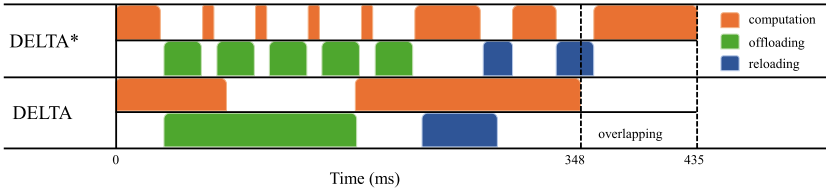


Fig. 13. Iteration timeline of DELTA and DELTA*.

Table 4. Comparison of the Swapping Cost (us) and the Recomputation Cost (us)

Tensor's name	c_s	c_r
BNForward (4034)	96,318	6
BNForward (6463)	22,230	4
BNForward (6526)	22,237	4
BNForward (6590)	11,148	4
ConvForward (96640)	22,805	69
PoolingForward (33219)	11,641	25

Table 5. Solving Time Comparison of POET and DELTA on Different Models

Models	DELTA	POET
ResNet-18	<100 ms	≈20 mins
ResNet-50	<100 ms	>24 hours
ViT-Large	<100 ms	>24 hours
BERT-Large	<100 ms	>24 hours
GPT2-Large	<1 s	>24 hours
GPT2-XL	<1 s	>24 hours

them when the batch size increases. This perspective supports the importance of prefetching in reducing time delay when we try to get some offloaded tensors back onto the GPU in the backward process.

7.6 Knapsack Algorithm and Heuristics

We compared the experimental results of different heuristic functions, including our own heuristic function, Equation (11), MSPS [38], LRU, and Greedy. Here, LRU is represented as $1/s(t)$, and Greedy is represented as $1/m(t)$. Since Capuchin has not been open-sourced and is implemented on the TensorFlow framework, it would be challenging for us to re-implement it. We have compared our heuristic function with their MSPS. In Figure 11, the bars represent the maximum batch size, and the points indicate the corresponding throughput. We observe that MSPS demonstrates better memory savings than LRU and Greedy, but has slightly lower throughput. For BERT-Large, GPT2-Large, and GPT2-XL, our heuristic function achieves optimal performance in terms of both the maximum batch size and the corresponding throughput. This outstanding performance is attributed to our memory-throughput joint optimization model and the corresponding design of heuristic functions.

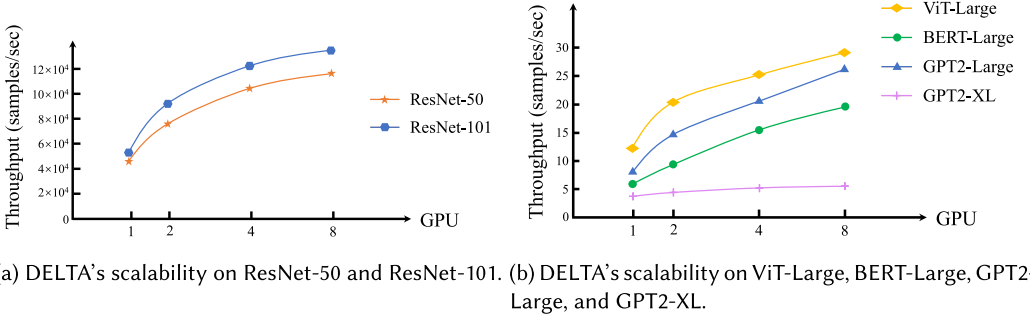


Fig. 14. Evaluation of DELTA's scalability.

Besides, we compare the Fractional Knapsack Greedy Algorithm with POET, an ILP-based method. As shown in Table 3, POET achieves more memory savings compared to DTR, but when compared to DELTA, it falls slightly short. Furthermore, as POET involves ILP modeling and utilizes optimization-solving tools to compute the optimal solution, utilizing POET for training models requires a substantial amount of time to first compute the behavior of corresponding tensors. In Table 5, we compared the search time for DELTA and POET across different models. As a result, POET needs to take more than 24 hours to solve the ILP for networks larger than ResNet-18, including ResNet-50, ViT-Large, BERT-Large, GPT2-Large, and GPT2-XL, while DELTA takes no more than 100 ms for ResNet-18, ResNet-50, and BERT-Large and no more than 1s for GPT2-Large and GPT2-XL. It demonstrates the advantage of DELTA in algorithm complexity.

7.7 Convergence

In this section, we conduct experiments on BERT-Large and GPT-Large to evaluate the loss comparisons with DELTA and Gradient Accumulation. Although gradient accumulation could also reduce memory consumption, it affects the variances of Batch Normalization, which is commonly used in vision models. As shown in Figure 12, the loss of DELTA is lower than the loss of the other two methods across all models, demonstrating the superiority of DELTA. To further ensure the correctness of DELTA, we focus on the convergence of DELTA in ResNet-50. The Vanilla process yields a final training loss on ImageNet of 1.5947. In comparison, DELTA achieves a training loss of 1.6054, which is nearly identical to that of the Vanilla, which provides evidence that DELTA nearly has the same performance as the Vanilla, proving the correctness of DELTA while gaining a promising memory reduction.

7.8 Distributed Training

Memory optimization is of great importance in distributed training [57]. In this subsection, we explore the scalability of DELTA in distributed training. In our experiments, we train ResNet-50, ResNet-101, ViT-Large, BERT-Large, GPT2-Large, and GPT2-XL with 8 A100 GPUs and evaluate the scalability of DELTA. ResNet-50 and ResNet-101 are trained with a batch size of 1024. For ViT-Large and BERT-Large, the batch size is 128, and for GPT2-Large and GPT2-XL, the batch size is 16. Figure 14 shows the curves of the training throughput of these models as the number of GPUs varies.

From Figure 14, we observe that for these models, DELTA exhibited good scalability, especially in the cases of ResNet-50, ResNet-101, ViT-Large, BERT-Large, and GPT2-Large. These experimental results not only demonstrate the effectiveness of DELTA in data-parallel training of models, but also confirm its good scalability as the number of GPUs increases. This is because DELTA only

Table 6. The Max Batch Size/throughput of Different Methods

Method	ResNet-50	ResNet-101	BERT-Large
DELTA	1814/82.34	1336/67.224	442/4.33
w/o swapping	1084/91.107	820/77.365	352/32.09
w/o recomputation	1890/31.554	1370/25.093	464/2.26

focuses on the data computation process during training. Data parallelism involves copying the model replica to all GPUs. DELTA and data parallelism are independent of each other. Even if we increase the number of GPUs, as long as the memory usage budget remains unchanged, the ideal situation is that the behavior of each tensor on each GPU is consistent. However, we also find that for the GPT2-XL model, DELTA's scalability is not as good as that of other models. We believe this is because the number of parameters in GPT2-XL is large, resulting in more memory redundancy and DELTA taking longer to compute the activation functions on each GPU, thereby leading to poorer scalability.

7.9 Ablation Study

In this section, we conduct an ablation study on the impact of swapping, considering the potential high cost incurred by CPU-GPU communication. The results are presented in Table 6.

In Table 6, “w/o swapping” means removing swapping in DELTA, where only recomputation is used. This case is similar to DTR, but with different heuristic functions. “w/o recomputation” means removing recomputation in DELTA. We observed that though “w/o swapping” achieves throughput improvement compared to DELTA, the maximum batch size decreases. However, “w/o recomputation” leads to a decrease in throughput but an increase in the maximum batch size. This is because the high cost of communication between the GPU and CPU results in a decrease in throughput. Meanwhile, the CPU's memory provides more storage space for tensor deallocation, thereby increasing the maximum batch size.

8 Conclusion

In this article, we propose DELTA, an automatic memory-efficient approach for DNN training. DELTA combines dynamic tensor swapping and dynamic tensor recomputation, which breaks the memory-saving limitation of current recomputation methods and alleviates the severe time delay. Moreover, to reduce the time delay, we utilize bidirectional prefetching to make communication and computation overlap as much as possible. With DELTA, we could optimize GPU memory utilization for deep learning systems automatically. Experimental results demonstrate the benefits of DELTA over Vanilla and DTR. DELTA saves up to about 40–72% of the GPU memory consumption for both CNN and language models. Besides, For GPT2-Large and GPT2-XL, DELTA gains an increased batch size of 5.5× and 6×, respectively. Overall, our work demonstrates the effectiveness of DELTA as a memory-efficient approach for training DNNs and opens up avenues for future research in this area.

References

- [1] Amir Gholami, Zhewei Yao, Sehoon Kim, Michael W. Mahoney, and Kurt Keutzer. 2021. AI and memory wall. <https://medium.com/riselab/ai-and-memory-wall-2cb4265cb0b8>. Accessed March 29, 2021.
- [2] Olivier Beaumont, Lionel Eyraud-Dubois, and Alena Shilova. 2021. Efficient combination of rematerialization and offloading for training DNNs. In *Advances in Neural Information Processing Systems*, Vol. 34. ACM, New York, NY.
- [3] Tarun Beri, Sorav Bansal, and Subodh Kumar. 2016. The Unicorn Runtime: Efficient distributed shared memory programming for hybrid CPU-GPU clusters. *IEEE Trans. Parallel Distrib. Syst.* 28, 5 (2016), 1518–1534.

- [4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, Vol. 33. ACM, New York, NY, 1877–1901.
- [5] Samuel Rota Buló, Lorenzo Porzi, and Peter Kotschieder. 2018. In-place activated batchnorm for memory-optimized training of dnn. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 5639–5647.
- [6] Cbc. 2023. COIN-OR Branch-and-Cut Solver. Retrieved February 5 from <https://github.com/coin-or/Cbc>, Last accessed on 2024-02-05.
- [7] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training deep nets with sublinear memory cost. arXiv:1604.06174. Retrieved from <https://arxiv.org/abs/1604.06174>
- [8] Sangjin Choi, Taeksoo Kim, Jinwoo Jeong, Rachata Ausavarungnirun, Myeongjae Jeon, Youngjin Kwon, and Jeongseob Ahn. 2022. Memory harvesting in Multi-GPU systems with hierarchical unified virtual memory. In *Proceedings of the USENIX Annual Technical Conference (USENIX ATC'22)*. 625–638.
- [9] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. 2020. An image is worth 16×16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*.
- [10] Krzysztof Dudziński and Stanisław Walukiewicz. 1987. Exact methods for the knapsack problem and its generalizations. *Eur. J. Operat. Res.* 28, 1 (1987), 3–21. [https://doi.org/10.1016/0377-2217\(87\)90165-2](https://doi.org/10.1016/0377-2217(87)90165-2)
- [11] Yanjie Gao, Yu Liu, Hongyu Zhang, Zhengxian Li, Yonghao Zhu, Haoxiang Lin, and Mao Yang. 2020. Estimating gpu memory consumption of deep learning models. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1342–1352.
- [12] gurobi. 2024. gurobi. Retrieved February 5, 2024 from <https://www.gurobi.com/>
- [13] Kai Han, Yunhe Wang, Hanjing Chen, Xinghao Chen, Jianyuan Guo, Zhenhua Liu, Yehui Tang, An Xiao, Chunjing Xu, Yixing Xu, et al. 2022. A survey on vision transformer. *IEEE Trans. Pattern Anal. Mach. Intell.* 45, 1 (2022), 87–110.
- [14] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 770–778.
- [15] Chien-Chin Huang, Gu Jin, and Jinyang Li. 2020. Swapadvisor: Pushing deep learning beyond the gpu memory limit via smart swapping. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems*. 1341–1355.
- [16] Sergey Ioffe and Christian Szegedy. 2015. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning*. PMLR, 448–456.
- [17] Hiroaki Ishii, Toshihide Ibaraki, and Hisashi Mine. 1977. Fractional knapsack problems. *Mathematical Programming* 13 (1977), 255–271.
- [18] Paras Jain, Ajay Jain, Aniruddha Nrusimha, Amir Gholami, Pieter Abbeel, Joseph Gonzalez, Kurt Keutzer, and Ion Stoica. 2020. Checkmate: Breaking the memory wall with optimal tensor rematerialization. In *Proceedings of Machine Learning and Systems*, Vol. 2 (2020), 497–511.
- [19] Jacob Devlin Ming-Wei Chang Kenton and Lee Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the Annual Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT'19)*. 4171–4186.
- [20] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. In *ICLR (Poster)*.
- [21] Marisa Kirisame, Steven Lyubomirsky, Altan Haan, Jennifer Brennan, Mike He, Jared Roesch, Tianqi Chen, and Zachary Tatlock. 2020. Dynamic tensor rematerialization. In *Proceedings of the International Conference on Learning Representations*.
- [22] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2012. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, Vol. 25. ACM, New York, NY, 1097–1105.
- [23] Ravi Kumar, Manish Purohit, Zoya Svitkina, Erik Vee, and Joshua Wang. 2019. Efficient rematerialization for deep networks. In *Advances in Neural Information Processing Systems*, Vol. 32. ACM, New York, NY.
- [24] Zhiquan Lai, Shengwei Li, Xudong Tang, Keshi Ge, Weijie Liu, Yabo Duan, Linbo Qiao, and Dongsheng Li. 2023. Merak: An efficient distributed dnn training framework with automated 3d parallelism for giant foundation models. *IEEE Trans. Parallel Distrib. Syst.* 34, 5 (2023), 1466–1478.
- [25] Matthew Leinhauser, Jeffrey Young, Sergei Bastrakov, René Widera, Ronnie Chatterjee, and Sunita Chandrasekaran. 2021. *Performance Analysis of PConGPU: Particle-in-Cell on GPUs using NVIDIA's NSight Systems and NSight Compute*. Technical Report. Oak Ridge National Laboratory.
- [26] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, et al. [n. d.]. PyTorch distributed: Experiences on accelerating data parallel training. *Proc. VLDB Endow.* 13, 12 ([n. d.]).
- [27] Yujun Lin, Song Han, Huizi Mao, Yu Wang, and Bill Dally. 2018. Deep gradient compression: Reducing the communication bandwidth for distributed training. In *International Conference on Learning Representations*.

- [28] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. 2021. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 10012–10022.
- [29] Brady D. Lund and Ting Wang. 2023. Chatting about ChatGPT: How may AI and GPT impact academia and libraries? *Library Hi Tech News* 40, 3 (2023), 26–29.
- [30] François Margot. 2010. Symmetry in integer linear programming. In *50 Years of Integer Programming 1958–2008* (2010), 647–686.
- [31] Silvano Martello and Paolo Toth. 1987. Algorithms for knapsack problems. In *Surveys in Combinatorial Optimization*, Silvano Martello, Gilbert Laporte, Michel Minoux, and Celso Ribeiro (Eds.). North-Holland Mathematics Studies, Vol. 132. North-Holland, Amsterdam, 213–257. [https://doi.org/10.1016/S0304-0208\(08\)73237-7](https://doi.org/10.1016/S0304-0208(08)73237-7)
- [32] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized pipeline parallelism for DNN training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. 1–15.
- [33] Xiaonan Nie, Xupeng Miao, Zhi Yang, and Bin Cui. 2022. TSPLIT: Fine-grained GPU memory management for efficient DNN training via tensor splitting. In *Proceedings of the IEEE 38th International Conference on Data Engineering (ICDE’22)*. IEEE, 2615–2628.
- [34] Nvidia. 2020. cudnn 8.0. Retrieved February, 5, 2024 from <https://developer.nvidia.com/rdp/cudnn-archive>
- [35] Nvidia. 2022. CUDA Toolkit 11.2. Retrieved February 5, 2024 from <https://developer.nvidia.com/cuda-toolkit>
- [36] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, Vol. 32, ACM, New York, NY, 8026–8037.
- [37] Shishir G. Patil, Paras Jain, Prabal Dutta, Ion Stoica, and Joseph Gonzalez. 2022. Poet: Training neural networks on tiny devices with integrated rematerialization and paging. In *Proceedings of the 39th International Conference on Machine Learning*. PMLR, 17573–17583.
- [38] Xuan Peng, Xuanhua Shi, Hulin Dai, Hai Jin, Weiliang Ma, Qian Xiong, Fan Yang, and Xuehai Qian. 2020. Capuchin: Tensor-based gpu memory management for deep learning. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems*. 891–905.
- [39] Bharadwaj Pudipeddi, Maral Mesmahkoshroshahi, Jinwen Xi, and Sujeeth Bharadwaj. 2020. Training large neural networks with constant memory using a new execution algorithm. arXiv:2002.05645. Retrieved from <https://arxiv.org/abs/2002.05645>
- [40] Günther R. Raidl and Jakob Puchinger. 2008. *Combining (Integer) Linear Programming Techniques and Metaheuristics for Combinatorial Optimization*. Springer, Berlin, 31–62. https://doi.org/10.1007/978-3-540-78295-7_2
- [41] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. ZeRO-infinity: Breaking the GPU memory wall for extreme scale deep learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC’21)*. Association for Computing Machinery, New York, NY, Article 59, 14 pages. <https://doi.org/10.1145/3458817.3476205>
- [42] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 2383–2392.
- [43] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 3505–3506.
- [44] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. ZeRO-Offload: Democratizing Billion-Scale model training. In *Proceedings of the USENIX Annual Technical Conference (USENIX ATC’21)*. 551–564.
- [45] Minsoo Rhu, Natalia Gimelshein, Jason Clemons, Arslan Zulfiqar, and Stephen W. Keckler. 2016. vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design. In *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO’16)*. IEEE, 1–13.
- [46] Corby Rosset. 2020. Turing-NLG: A 17-billion-parameter language model by Microsoft. *Microsoft Blog* 1, 2 (2020).
- [47] Sartaj Sahni. 1975. Approximate algorithms for the 0/1 knapsack problem. *J. ACM* 22, 1 (1975), 115–124.
- [48] Harvey M. Salkin and Cornelis A. De Kluyver. 1975. The knapsack problem: A survey. *Nav. Res. Logist. Quart.* 22, 1 (1975), 127–144.
- [49] Johannes Schmidt-Hieber. 2020. Nonparametric regression using deep neural networks with ReLU activation function. *Ann. Stat.* 48, 4 (2020), 1875–1897.
- [50] Manuela Schuler, Richard Membarth, and Philipp Slusallek. 2022. Xengine: Optimal tensor rematerialization for neural networks in heterogeneous environments. *ACM Trans. Arch. Code Optimiz.* 20, 1 (2022), 1–25.

- [51] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. arXiv:1909.08053. Retrieved from <https://arxiv.org/abs/1909.08053>
- [52] Christian Szegedy, Sergey Ioffe, Vincent Vanhoucke, and Alexander A. Alemi. 2017. Inception-v4, inception-resnet and the impact of residual connections on learning. In *Proceedings of the 31st AAAI Conference on Artificial Intelligence*.
- [53] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*. ACM, New York, NY, 5998–6008.
- [54] Linnan Wang, Jinmian Ye, Yiyang Zhao, Wei Wu, Ang Li, Shuaiwen Leon Song, Zenglin Xu, and Tim Kraska. 2018. Superneurons: Dynamic GPU memory management for training deep neural networks. In *Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 41–53.
- [55] Qihan Wang, Zhen Peng, Bin Ren, Jie Chen, and Robert G. Edwards. 2022. MemHC: An optimized GPU memory management framework for accelerating many-body correlation. *ACM Trans. Arch. Code Optim.* 19, 2 (2022), 1–26.
- [56] Yuzhong Wang, Xu Han, Weilin Zhao, Guoyang Zeng, Zhiyuan Liu, and Maosong Sun. 2023. H3T: Efficient integration of memory optimization and parallelism for large-scale transformer training. In *Proceedings of the 37th Conference on Neural Information Processing Systems*.
- [57] Yuzhong Wang, Xu Han, Weilin Zhao, Guoyang Zeng, Zhiyuan Liu, and Maosong Sun. 2024. H3T: Efficient integration of memory optimization and parallelism for large-scale transformer training. In *Advances in Neural Information Processing Systems*, Vol. 36. ACM, New York, NY.
- [58] Jia Wei, Xingjun Zhang, Longxiang Wang, and Zheng Wei. 2023. Fastensor: Optimise the tensor I/O path from SSD to GPU for deep learning training. *ACM Trans. Arch. Code Optim.* (2023).
- [59] Jianhao Zhang, Shihan Ma, Peihong Liu, and Jinhui Yuan. 2023. Coop: Memory is not a Commodity. In *Proceedings of the 37th Conference on Neural Information Processing Systems*.
- [60] Yu Zhang, Da Peng, Xiaofei Liao, Hai Jin, Haikun Liu, Lin Gu, and Bingsheng He. 2021. LargeGraph: An efficient dependency-aware GPU-accelerated large-scale graph processing. *ACM Trans. Arch. Code Optim.* 18, 4 (2021), 1–24.
- [61] Hongyu Zhu, Mohamed Akrouf, Bojian Zheng, Andrew Pelegrini, Amar Phanishayee, Bianca Schroeder, and Gennady Pekhimenko. 2018. Tbd: Benchmarking and analyzing deep neural network training. arXiv:1803.06905. Retrieved from <https://arxiv.org/abs/1803.06905>

Received 5 February 2024; revised 1 June 2024; accepted 6 August 2024